Mayini Framework (v0.8.1) - The Ultimate Code Cookbook

Welcome to the exhaustive code reference for the Mayini Framework. This document contains code snippets demonstrating EVERY major feature, initialization technique, layer, loss function, and optimization method across the entire ecosystem.

1. TENSORS, NEURAL NETWORKS, AND REGULARIZATION

The complete deep learning module (mayini.nn) is equipped with dropouts, layer architectures, batch normalization, and all key activations mapping down to our native float64 Autograd Engine.

1A. Complete Neural Network Architecture (Conv2D, RNN, GRU, LSTM, Regularization)

```
from mayini.tensor import Tensor
import mayini.nn as nn
import numpy as np

# A deeply complex architecture combining Vision, Sequence, and Fully Connected operations
class UltimateArchitecture(nn.Module):
    def __init__(self):
        super().__init__()

        # --- VISION & CONVOLUTION BLOCK ---
        self.vision_block = nn.Sequential(
            nn.Conv2D(in_channels=3, out_channels=16, kernel_size=3, padding=1),
            nn.BatchNorm1d(num_features=16),
            nn.LeakyReLU(alpha=0.01),
            nn.MaxPool2D(kernel_size=2, stride=2),
            nn.Dropout(p=0.25),

            nn.Conv2D(in_channels=16, out_channels=32, kernel_size=3, padding=1),
            nn.GELU(),
            nn.AvgPool2D(kernel_size=2, stride=2),
            nn.Dropout(p=0.4),
            nn.Flatten()
        )

        # --- SEQUENCE MODELING BLOCK (Recurrent Cells) ---
        # Note: Mayini supports nn.RNNCell, nn.LSTMCell, nn.GRUCell, and native nn.RNN wrapper
        self.lstm = nn.LSTMCell(input_size=128, hidden_size=64)

        # --- CLASSIFIER / REGRESSION BLOCK ---
        self.classifier = nn.Sequential(
            nn.Linear(in_features=64, out_features=128),
            nn.ReLU(),
            nn.Dropout(p=0.5), # Regularization via 50% random unweighting
            nn.Linear(in_features=128, out_features=10),
            nn.Softmax()       # Softmax for multi-class probability extraction
        )

    def forward(self, x_image, x_sequence):
        # 1. Process Spatial Data
        features = self.vision_block(x_image)

        # 2. Process Temporal Data Step-by-Step with LSTM Cell
        hidden_state = Tensor(np.zeros((x_sequence.shape[0], 64)))
        cell_state = Tensor(np.zeros((x_sequence.shape[0], 64)))

        for t in range(x_sequence.shape[1]):
            hidden_state, cell_state = self.lstm(x_sequence[:, t, :], (hidden_state, cell_state))

        # 3. Combine & Predict
        combined = features + hidden_state # Native Autograd Broadcasting
        return self.classifier(combined)
```

Mayini Framework (v0.8.1) - The Ultimate Code Cookbook

1B. Exhaustive Optimizers and Loss Functions Tracker

```
from mayini.optim import Adam, RMSprop, SGD
from mayini.nn.losses import MSELoss, MAELoss, CrossEntropyLoss, BCELoss, HuberLoss

model = UltimateArchitecture()

# -------- ALL SUPPORTED OPTIMIZERS --------
opt_sgd = SGD(model.parameters(), lr=0.01, momentum=0.9, weight_decay=1e-4)
opt_rmsprop = RMSprop(model.parameters(), lr=0.001, alpha=0.99, eps=1e-8)
opt_adam = Adam(model.parameters(), lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8)

# -------- ALL SUPPORTED LOSS FUNCTIONS --------
criterion_mse = MSELoss()             # Regression: Basic Mean Squared Error
criterion_mae = MAELoss()             # Regression: Mean Absolute Error (L1 Loss)
criterion_huber = HuberLoss(delta=1.0) # Regression: Huber Loss (Robust L1/L2)
criterion_bce = BCELoss()             # Classification: Binary Cross Entropy
criterion_ce = CrossEntropyLoss()     # Classification: Multi-class

model.eval()   # Evaluation Mode Trigger (Disables Dropout & Freezes BatchNorm)
model.train()  # Training Mode Trigger
```

Mayini Framework (v0.8.1) - The Ultimate Code Cookbook

2. CLASSICAL MACHINE LEARNING FULL SPECTRUM

Providing everything from native SVM hyperplanes, impurity evaluations via Gini/Entropy in Decision Trees, to ensemble aggregation techniques.

2A. Supervised & Unsupervised Operations

```
from mayini.ml.supervised.tree_models import DecisionTree
from mayini.ml.supervised.naive_bayes import GaussianNB
from mayini.ml.supervised.knn import KNN
from mayini.ml.supervised.svm import SVM
from mayini.ml.supervised.linear_models import LinearRegression, LogisticRegression
from mayini.ml.unsupervised.clustering import KMeans
from mayini.ml.unsupervised.decomposition import PCA
from mayini.ml.unsupervised.anomaly import IsolationForest

# --- Supervised Regression & Classification ---
tree = DecisionTree(max_depth=5, min_samples_split=2)
svm = SVM(learning_rate=0.01, lambda_param=0.01, n_iters=1000)
lr = LinearRegression(lr=0.001, n_iters=1000)
log_reg = LogisticRegression(lr=0.01)

# --- Unsupervised Methods ---
pca = PCA(n_components=3)
kmeans = KMeans(k=5, max_iters=300)
iso_forest = IsolationForest(n_trees=150, max_samples=512) # Anomaly Isolation
```

2B. The Ensemble Module (Aggregating Weak Learners)

```
from mayini.ml.ensemble.bagging import RandomForest
from mayini.ml.ensemble.boosting import AdaBoost
from mayini.ml.ensemble.voting import VotingClassifier

# Bagging (Parallelization)
rf = RandomForest(n_estimators=100, max_depth=10, min_samples_split=3)
rf.fit(X_train, y_train)

# Boosting (Sequential Error Correction) - Adaptive Boosting
adaboost = AdaBoost(n_estimators=50, learning_rate=0.5)
adaboost.fit(X_train, y_train)

# Voting Ensembles (Aggregating multiple different models based on hard predictions)
voter = VotingClassifier(estimators=[rf, adaboost, svm, tree], voting='hard')
voter.fit(X_train, y_train)
best_preds = voter.predict(X_test)
```

Mayini Framework (v0.8.1) - The Ultimate Code Cookbook

3. ADVANCED NEUROEVOLUTION (NEAT) CONFIGURATIONS

Explore massive possible formations: Speciation mechanics, structural topologies, cross-over environments, and tracking innovations via genetic mutation loops.

3A: Building Topologies: Explicit Genomes & Mutations

```
from mayini.neat.config import Config
from mayini.neat.population import Population
from mayini.neat.evaluator import XORFitnessEvaluator
from mayini.neat.genome import Genome
from mayini.neat.innovation import InnovationTracker

# --- COMPREHENSIVE SETTINGS FOR NEAT ---
config = Config(
    pop_size=500,                       # Massive population sizes supported
    num_inputs=10,                      # Sensor nodes
    num_outputs=3,                      # Action nodes
    compatibility_threshold=3.5,        # Threshold to declare a new Species (protecting genetics)
    c1=1.0, c2=1.0, c3=0.4,             # Speciation distance coefficients (disjoint, excess, weight)
    weight_mutation_power=0.6,          # Rate of weight shift logic
    weight_mutate_prob=0.8,             # 80% chance to perturb weights
    weight_replace_prob=0.1,            # 10% chance to completely override a weight
    node_add_prob=0.03,                 # Probability of structural neuron additions
    conn_add_prob=0.15,                 # Probability of synaptic attachment additions
    survival_threshold=0.2              # Only top 20% of species survive to reproduce
)

# --- MANUAL GENOME TOPOLOGY MANIPULATION ---
innovation_tracker = InnovationTracker()
genome = Genome(id=99, config=config)

# Manually configuring standard Feed-Forward logic
genome.add_node(node_id=11, node_type="hidden", activation="relu")
genome.add_node(node_id=12, node_type="hidden", activation="tanh")
genome.add_node(node_id=13, node_type="hidden", activation="sigmoid")

# Innov_id tracks matching genes across different parents for crossover mating
genome.add_connection(in_node=0, out_node=11, weight=1.1, innov_id=1)
genome.add_connection(in_node=11, out_node=12, weight=-0.5, innov_id=2)
genome.add_connection(in_node=12, out_node=13, weight=2.0, innov_id=3)
genome.add_connection(in_node=13, out_node=1, weight=0.8, innov_id=4)

# Force a mutation event mathematically
genome.mutate(innovation_tracker)

# Activate network manually
from mayini.neat.network import FeedForwardNetwork
nn_topology = FeedForwardNetwork.create(genome, config)
outputs = nn_topology.activate([1.0] * 10) # Pass inputs
```

3B: Population & Speciation Lifecycle Tracking

```
# --- RUNNING REAL EVOLUTION CYCLES ---
# Speciation categorizes identical structures to preserve genetic safety.
evaluator = XORFitnessEvaluator()
population = Population(config)
best_network = population.run(evaluator.evaluate, n_generations=1000)

for species_id, species in population.species.items():
    print(f"Species {species_id} -> Fitness: {species.fitness}, Members: {len(species.members)}")
```

Mayini Framework (v0.8.1) - The Ultimate Code Cookbook

4. AUTOMATED PREPROCESSING: EVERY PARAMETER SHOWN

An exhaustive list of the multimodal processing capabilities you can stack on top of your DataLake prior to inference.

4A. Advanced Text & NLP Feature Engineering

```
from mayini.preprocessing import AutomatedPreprocessor
from mayini.preprocessing.widget import launch_widget

txt_prep = AutomatedPreprocessor()
txt_prep.load_data("corpus.txt", modality="text")
txt_pipe = [
    # NLP Cleaning: Strips Punctuation, Emails, HTML cleanly
    {'operation': 'clean', 'params': {
        'remove_urls': True, 'remove_emails': True,
        'lowercase': True, 'remove_punctuation': True
    }},
    # Tokenization supports WORD and SENTENCE
    {'operation': 'tokenize', 'params': {'type': 'word'}},

    # Stemmers: 'porter' and 'lancaster' supported natively
    {'operation': 'stem', 'params': {'stemmer': 'lancaster'}},

    # Vectorizers: TF-IDF, Count Vectors, Word2Vec
    {'operation': 'vectorize', 'params': {'type': 'tfidf', 'max_features': 15000}}
]
```

4B. Deep Image Alterations & Scene Detection

```
img_prep = AutomatedPreprocessor()
img_prep.load_data("medical_scan.png", modality="image")
img_pipe = [
    {'operation': 'resize', 'params': {'size': (512, 512), 'method': 'bilinear'}},
    # Complex Augmentations built for deep learning dataloader generators
    {'operation': 'augment', 'params': {
        'noise': True, 'brightness': True,
        'contrast': 1.5, 'saturation': 0.8, 'jitter': 0.1
    }},
    # Feature Extractions: Sobel, Canny, HOG
    {'operation': 'edge_detection', 'params': {'method': 'canny', 'threshold1': 100, 'threshold2': 200}}
]

# --- VIDEO BOUNDING BOXES & SCENE DETECTION ---
vid_prep = AutomatedPreprocessor()
vid_prep.load_data("robotics_cam.mp4", modality="video")
vid_pipe = [
    # Scene detection automatically extracts cuts and transitions
    {'operation': 'scene_detection', 'params': {'method': 'content', 'threshold': 27.0}}
]
```

4C. Audio Waveform Shifting & Spectrogram Output

```
audio_prep = AutomatedPreprocessor()
audio_prep.load_data("speech.wav", modality="audio")

audio_pipe = [
    # Waveform DSP Modifiers
    {'operation': 'effects', 'params': {'effect': 'reverb', 'room_size': 0.8}},
    {'operation': 'pitch_shift', 'params': {'semitones': 2, 'bins_per_octave': 12}},

    # Mathematical Audio Analysis conversions
```

```
    {'operation': 'mfcc', 'params': {'n_mfcc': 40, 'n_fft': 2048, 'hop_length': 512}},
    # Or generate a full Mel-Spectrogram:
    # {'operation': 'spectrogram', 'params': {'log_scale': True}}
]

# Process and map back to memory for tensor usage
audio_tensors = audio_prep.preprocess(audio_pipe)

# -------- THE GRADIO NO-CODE PIPELINE BUILDER --------
# Provides a completely interactive web-canvas for ALL operations above.
launch_widget(share=True)
```